Serial Python Minimalis: Tutorial #03-List dan Loop

Ahmad R. T. Nugraha

ver. 21 April 2026

1 Definisi, Sintaksis, dan Penggunaan

Untuk memahami konsep `list` dan `loop`, kita kembali pada konsep string yang dipelajari dalam tutorial sebelumnya. Pada dasarnya, string adalah salah satu contoh barisan (*sequence*) atau kumpulan objek (dalam hal ini, karakter) yang dapat diambil satu per satu (alias di-"iterasi"). Sintaksis untuk mengambil objek dari sebuah barisan adalah:

```
for objek in barisan:
    # Lakukan sesuatu dengan objek
```

Bentuk ini merupakan perulangan `for` (*for loop*). Dalam bahasa Python, blok kode yang diindentasi dengan empat spasi dieksekusi satu kali untuk setiap objek dalam barisan tersebut. Contohnya:

```
name = 'tin'
for letter in name:
    print(letter)
```

Output:

```
t
i
n
```

Bentuk barisan Python penting lainnya adalah `list`, yaitu kumpulan objek terurut yang dapat berupa tipe apa pun. `list` didefinisikan sebagai barisan objek yang dipisahkan koma di antara tanda kurung siku:

```
mylist = [4, 'Sn', 'sodium', 3.14159]
```

`list` dapat diindeks, dipotong (*sliced*), dan diiterasi sama seperti string:

```
print(mylist[2])
```

Output:

```
sodium
```

```
# Semua item mulai dari indeks ketiga dan seterusnya
print(mylist[2:])
```

Output:

```
['sodium', 3.14159]
```

```
# Mengiterasi semua item dalam mylist
for item in mylist:
    print(item)
```

Output:

```
4
Sn
sodium
3.14159
```

Tidak seperti string, `list` bersifat *mutable*, yakni dapat diubah di tempat (*in place*) dengan berbagai cara, misalnya melalui penetapan langsung:

```
mylist[2] = 'potassium'
mylist
```

Output:

```
[4, 'Sn', 'potassium', 3.14159]
```

Ada beberapa metode penting lainnya untuk memanipulasi `list`. Metode `append` menambahkan sebuah item ke akhir list (menambah panjangnya sebanyak satu):

```
mylist.append(-42)
mylist
```

Output:

```
[4, 'Sn', 'sodium', 3.14159, -42]
```

Metode `extend` dapat digunakan untuk menggabungkan dua `list`:

```
list1 = ['H', 'He', 'Li']
list2 = ['Be', 'B', 'C']
list1.extend(list2)
list1
```

Output:

```
['H', 'He', 'Li', 'Be', 'B', 'C']
```

`list` baru dapat dibuat dengan melewatkan objek teriterasi (*iterable*) apa pun ke dalam konstruktor `list()`:

```
list('helium')
```

Output:

```
['h', 'e', 'l', 'i', 'u', 'm']
```

Angka individual tidak *iterable*, sehingga `list(3.14159)` akan memberikan pesan *error*:

```
list(3.14159)
```

Output:

```
TypeError                                 Traceback (most recent call last)
<ipython-input-32-8870af92a665> in <module>
----> 1 list(3.14159)

TypeError: 'float' object is not iterable
```

Namun, fungsi `str(3.14159)` yang merupakan representasi string dari angka ini, "3.14159", bersifat *iterable* sehingga kita dapat membuat `list` dari karakter-karakternya:

```
list(str(3.14159))
```

Output:

```
['3', '.', '1', '4', '1', '5', '9']
```

2 Fungsi `range` dan `enumerate`

Untuk menghasilkan barisan bilangan bulat dengan jarak yang teratur (sebuah deret aritmetika), terdapat fungsi bawaan yang disebut `range`. Dengan argumen tunggal, `range(n)` menghasilkan bilangan bulat 0, 1, 2, ... n-1:

```
for i in range(4):
    print(i, i**2, i**3)
```

Output:

```
0 0 0
1 1 1
2 4 8
3 9 27
```

Perhatikan bahwa barisan ini mengikuti aturan umum dimulai dari 0 dan berakhir pada $n - 1$ sehingga cocok digunakan untuk menghasilkan indeks pada urutan lain seperti `list` dan string. Ingat kembali contoh iterasi karakter dalam sebuah string:

```
for letter in 'tin':
    print(letter)
```

Output:

```
t
i
n
```

Contoh di sini menunjukkan ekspresifnya Python sebagai bahasa pemrograman. Dalam bahasa pemrograman lain, kita perlu melacak indeks ke dalam barisan dan menggunakannya untuk mengambil item satu per satu, sesuatu yang dapat langsung dicapai dengan fungsi `range`:

```
name = 'tin'
for i in range(len(name)):
    # yaitu, i = 0, 1, 2
    print(name[i])
```

Output:

```
t
i
n
```

Meski begitu, penggunaan `range` semacam ini tidak dianjurkan di Python karena ada sintaksis `for letter in name:` yang lebih sederhana dan jelas lebih mudah dipahami dan dikelola. Jika ada alasan tertentu yang membuat kita menginginkan indeks dari setiap item serta item itu sendiri, terdapat fungsi bawaan lainnya untuk keperluan tersebut, yaitu `enumerate`, yang mengembalikan keduanya dan dapat diekstrak (*unpacked*) menjadi dua variabel di dalam perulangan:

```
for i, letter in enumerate('zinc'):
    print(i, ':', letter)
```

Output:

```
0 : z
1 : i
2 : n
3 : c
```

3 Membuat `list`

Karena `list` dapat dikembangkan di tempat, salah satu cara untuk membuatnya adalah dengan menambahkan (`append`) elemen-elemennya satu per satu:

```
my_list = []
for i in range(5):
    my_list.append((i + 1)**2)
my_list
```

Output:

```
[1, 4, 9, 16, 25]
```

Namun, terdapat sintaksis yang lebih ringkas dan umum digunakan dalam kasus ini yang disebut *list comprehension*:

```
my_list = [(i + 1)**2 for i in range(5)]
my_list
```

Output:

```
[1, 4, 9, 16, 25]
```

Ekspresi yang cukup kompleks dapat digunakan dalam *list comprehension*, tetapi sebaiknya pastikan bahwa kode tersebut tetap mudah dipahami. Untungnya, Python cukup ekspresif sehingga beberapa fungsi dapat dirangkai tanpa kehilangan keterbacaan. Misalnya, untuk menghitung $\sin(\theta)$ pada beberapa sudut, θ:

```
import math
angles = [0, 30, 60, 90, 120, 150, 180]
sines = [round(math.sin(math.radians(theta)), 3) for theta in angles]
sines
```

Output:

```
[0.0, 0.5, 0.866, 1.0, 0.866, 0.5, 0.0]
```

4 Metode `split` dan `join`

Dua metode berguna ini bekerja pada string. Untuk memecah string menjadi `list` yang berisi substring berdasarkan pemisah (*separator*) tertentu, panggil `split()`:

```
'a,b,c'.split(',')
```

Output:

```
['a', 'b', 'c']
```

```
'a::b::c'.split(':')
```

Output:

```
['a', '', 'b', '', 'c']
```

```
'a::b::c'.split('::')
```

Output:

```
['a', 'b', 'c']
```

Jika kita ingin memecahnya berdasarkan spasi kosong dalam jumlah berapa pun (spasi, tab), tidak perlu memberikan argumen pada `split()`:

```
'a b   c'.split()
```

Output:

```
['a', 'b', 'c']
```

Metode `join` melakukan hal sebaliknya, yakni menggabungkan urutan string dengan pemisah (*separator*) yang ditentukan:

```
'-'.join(['a', 'b', 'c'])
```

Output:

```
'a-b-c'
```

Urutan tersebut tidak harus berupa `list`. String adalah urutan karakter (yang dengan sendirinya adalah string), sehingga hal berikut juga berlaku:

```
'-'.join('abc')
```

Output:

```
'a-b-c'
```

5 Fungsi `zip`

Terkadang kita perlu mengiterasi dua urutan atau lebih pada saat yang bersamaan, yang dapat dilakukan dengan pengindeksan langsung ke dalamnya. Sebagai contoh:

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
for i in range(len(list1)):
    print(list1[i], list2[i])
```

Output:

```
1 a
2 b
3 c
```

Fungsi bawaan `zip` memungkinkan iterasi tanpa pengindeksan eksplisit. Saat diulang, fungsi ini akan mengeluarkan item-item yang cocok dari setiap urutan secara bergantian:

```
for number, letter in zip(list1, list2):
    print(number, letter)
```

Output:

```
1 a
2 b
3 c
```

Jika urutan-urutan tersebut memiliki panjang yang berbeda, `zip` akan mengembalikan elemen-elemen yang cocok sampai urutan yang paling pendek habis:

```
seq1 = 'abcdefghijk'
seq2 = range(1000)
seq3 = ['Be', 'Mg', 'Ca', 'Sr']
for letter, number, symbol in zip(seq1, seq2, seq3):
    print(letter, number, symbol)
```

Output:

```
a 0 Be
b 1 Mg
c 2 Ca
d 3 Sr
```

Studi Kasus

Kasus 1 Simbol dan Nomor Atom

Buatlah tabel yang berisi 10 simbol unsur pertama beserta nomor atomnya, Z.

Solusi

Karena indeks Python dimulai dari 0, kita harus menambahkan 1 pada indeks urutan `list` simbol unsur untuk mendapatkan Z:

```python
symbols = ['H', 'He', 'Li', 'Be', 'B', 'C', 'N', 'O', 'F', 'Ne']

print('Element | Z')
print('--------|---')
for Z, symbol in enumerate(symbols):
    print('{:7s} | {:2d}'.format(symbol, Z + 1))
```

Output:

```
Element | Z
--------|---
H       |  1
He      |  2
Li      |  3
Be      |  4
B       |  5
C       |  6
N       |  7
O       |  8
F       |  9
Ne      | 10
```

Sebagai alternatif, `enumerate` dapat menerima argumen opsional, `start`, yang menetapkan bilangan bulat pertama untuk dikembalikan sebagai hitungan awal:

```python
for Z, symbol in enumerate(symbols, start=1):
    print('{:7s} | {:2d}'.format(symbol, Z))
```

Kasus 2 Massa Molar Udara Kering

Hitunglah rata-rata massa molar udara kering, berdasarkan komposisi dan massa yang tercantum pada Tabel 4.1.

Molekul	Massa / Da	Komposisi / %
N_2	28.0134	78.084
O_2	31.9898	20.946
Ar	39.948	0.9290
CO_2	44.0095	0.041

Solusi

Kita mendefinisikan data ke dalam `list`. Kemudian, kita perlu mengiterasi dua `list` secara bersamaan menggunakan `zip` untuk menghitung rata-ratanya:

$$\overline{M}_{air} = \sum_i x_i M_i$$

dengan x_i adalah fraksi jumlah volume dari konstituen udara ke-i, dengan massa molar M_i.

```
# Konstituen udara kering:
# molekul, massa (g.mol-1), persentase komposisi volume
molecule = ['N2', 'O2', 'Ar', 'CO2']
mass = [28.0134, 31.9898, 39.948, 44.0095]
composition = [78.084, 20.946, 0.9290, 0.041]

mean_air_mass = 0
for m, x_perc in zip(mass, composition):
    mean_air_mass += m * x_perc / 100

print('Mean mass of dry air: {:.2f} g.mol-1'.format(mean_air_mass))
```

Output:

```
Mean mass of dry air: 28.96 g.mol-1
```

Kasus 3 Gaya Apung Balon

Sebuah balon, dengan massa 0.8 g saat belum ditiup, diisi dengan gas helium hingga bervolume 2 L pada tekanan lingkungan 1 atm. Berapakah gaya netto (*net force*) pada balon pada suhu-suhu berikut: $-20, 0, 25, 40°$C?
Ambil $M(\text{He}) = 4$ g.mol^{-1}, massa molar rata-rata udara $\overline{M}_{air} = 29$ g.mol^{-1}, dan percepatan gravitasi $g = 9.8$ m.s^{-2}.

Solusi

Memperlakukan helium sebagai gas ideal, jumlah molnya adalah

$$n = pV/RT$$

dengan tekanan, p, dapat dianggap sama dengan tekanan lingkungan (mengabaikan efek kelengkungan permukaan balon yang mengembang).
Ada dua gaya pada balon: (1) beratnya, $F_g = -mg$, dengan $m = m_{balloon} + m_{He}$, dan (2) gaya apung karena udara yang dipindahkan, $F_b = m_{air}g$. Gaya nettonya adalah $F = F_g + F_b$ dengan nilai negatif F menunjukkan balon tenggelam (gaya

diarahkan ke bawah), dan nilai positif menunjukkan balon naik.
Pertama, definisikan variabel untuk konstanta dan parameter masalah ini:

```
# Konstanta gas (J.K-1.mol-1) dan percepatan gravitasi (m.s-2).
R = 8.314
g = 9.8
# Tekanan (Pa).
p = 101325
# Volume balon mengembang (m3).
V = 2.e-3
# Massa balon belum ditiup (kg).
m_balloon = 0.8 / 1000
# Massa molar helium dan rata-rata massa molar udara (g.mol-1).
M_He = 4
M_air = 29

# Suhu dalam degC
temperatures = [-20, 0, 25, 40]
```

Sekarang perhitungan gaya netto dapat dilakukan di dalam perulangan untuk berbagai suhu:

```
for T in temperatures:
    # Jumlah (mol) dan massa (kg) He dalam balon.
    n_He = p * V / R / (T + 273)
    m_He = M_He / 1000 * n_He

    # Udara yang dipindahkan memiliki jumlah mol yang sama, tapi lebih
    ↪ berat.
    m_air = M_air / 1000 * n_He

    # Gaya netto
    F = (m_air - m_He - m_balloon) * g
    print('{:3d} deg C : {:.4f} N'.format(T, F))
```

Output:

```
-20 deg C : 0.0158 N
  0 deg C : 0.0140 N
 25 deg C : 0.0122 N
 40 deg C : 0.0112 N
```

Perulangan bisa dibuat bersarang (*nested*). Misalnya, kita dapat mengulangi perhitungan tersebut untuk gas Ne dan Ar juga:

```python
# Gas dan massa molarnya
gases = ['He', 'Ne', 'Ar']
gas_masses = [4, 20, 40]

for T in temperatures:
    n = p * V / R / (T + 273)
    m_air = M_air / 1000 * n

    for gas, M_gas in zip(gases, gas_masses):
        m_gas = M_gas / 1000 * n
        F = (m_air - m_gas - m_balloon) * g
        print('{} at {:3d} deg C: Force = {:8.5f} N'.format(gas, T, F))
```

Output:

```
He at -20 deg C: Force =  0.01576 N
Ne at -20 deg C: Force =  0.00066 N
Ar at -20 deg C: Force = -0.01823 N
He at   0 deg C: Force =  0.01403 N
Ne at   0 deg C: Force =  0.00003 N
Ar at   0 deg C: Force = -0.01746 N
He at  25 deg C: Force =  0.01220 N
Ne at  25 deg C: Force = -0.00063 N
Ar at  25 deg C: Force = -0.01666 N
He at  40 deg C: Force =  0.01124 N
Ne at  40 deg C: Force = -0.00097 N
Ar at  40 deg C: Force = -0.01623 N
```

Dengan sedikit modifikasi, kita dapat menggunakan `print` untuk menghasilkan tabel dari hasil tersebut. Gunakan parameter `end=''` untuk menekan perilaku *default* `print` yang selalu menambahkan baris baru di akhir.

```python
print('Net force on balloon (in newtons; +ve is up)')
print('  T      He          Ne          Ar')

for T in temperatures:
    n = p * V / R / (T + 273)
    m_air = M_air / 1000 * n
    print('{:3d} °C'.format(T), end='')

    for gas, M_gas in zip(gases, gas_masses):
        m_gas = M_gas / 1000 * n
        F = (m_air - m_gas - m_balloon) * g
        print('{:12.5f}'.format(F), end='')

    # Mencetak baris baru untuk suhu berikutnya
    print()
```

Output:

```
Net force on balloon (in newtons; +ve is up)
  T      He          Ne          Ar
-20 C     0.01576     0.00066    -0.01823
  0 C     0.01403     0.00003    -0.01746
 25 C     0.01220    -0.00063    -0.01666
 40 C     0.01124    -0.00097    -0.01623
```